

CSI/SE/SS ZG527 — Cloud Computing

Containers Module — Live Demo Script

Instructor reference. All commands assume a single shared AWS EC2 instance (Ubuntu 24.04 LTS) unless otherwise noted.

Overview and Sequencing

This script covers four demos in sequence, run on the EC2 instance already used for the LXD/Docker exercises earlier in the course, with one exception noted explicitly in Part 4.

- Part 1 — OS Containers using LXD
- Part 2 — Docker basics (install, build a simple image, detached vs. foreground)
- Part 3 — Dockerfile demo (official image vs. custom Dockerfile, PostgreSQL, with external access from a laptop)
- Part 4 — Docker Desktop demo (navigation walkthrough) — requires a pre-check before attempting on this EC2 instance.

Part 1 — OS Containers Demo Using LXD

Goal: Demonstrate that a container shares the host kernel but can run a completely different Linux distribution in its userland — same kernel, different OS.

1.1 Launch the AWS EC2 Instance

- AMI: Ubuntu 24.04 LTS
- Instance type: t3.small (LXD's snap install and first image pull need more headroom than t3.micro/t2.micro comfortably give)
- Security group: inbound SSH (port 22) only, for this part

```
ssh -i yourkey.pem ubuntu@<ec2-public-ip>
```

1.2 Log In and Install LXD

Step 1: Check whether LXD is already installed via snap

```
if snap list lxd &>/dev/null; then
  echo "LXD is already installed."
  snap list lxd
else
  echo "LXD not found. Proceeding to install."
fi
```

Run this before every session on a reused instance, not just the first time — it prevents re-running snap install lxd unnecessarily.

Step 2: Install LXD via snap (only if the check above reports not found)

```
sudo snap install lxd
```

Step 3: Initialize LXD (minimal, non-interactive) — first-time setup only

```
sudo lxd init --minimal
```

Step 4: Check whether the LXD daemon is actually running

```
if systemctl is-active --quiet snap.lxd.daemon; then
  echo "LXD daemon is running."
else
  echo "LXD daemon is NOT running. Investigate before proceeding:"
  systemctl status snap.lxd.daemon
fi
```

Do not proceed to 1.3 if this reports the daemon is not running — every lxc command depends on this daemon being active, and a failed lxc launch here is a symptom of this check failing, not a separate problem.

1.3 Launch a Different Linux Distribution (Not Ubuntu)

The host is Ubuntu. Launch a Rocky Linux container to make the kernel-vs-distro distinction concrete.

Step 1: Verify the image registry before launching

```
lxc remote list
lxc image list images: | grep -i rocky
lxc image info rockylinux/9
```

`lxc remote list` confirms which URL the `images: remote` actually resolves to on this specific LXD install — worth checking rather than assuming, since this changed after the 2023 LXD/Incus project split (covered earlier). `lxc image list images:` shows every image available from that remote; `grep` narrows it to the Rocky Linux entries specifically. `lxc image info` prints metadata for one image — size, fingerprint, aliases — before committing to a launch.

Step 2: Check whether `os-demo-rocky` already exists

```
if lxc list --format csv -c n | grep -qx "os-demo-rocky"; then
  echo "os-demo-rocky already exists:"
  lxc list os-demo-rocky
else
  echo "Not found. Proceeding to launch."
  lxc launch images:rockylinux/9 os-demo-rocky
fi
```

Caution: The `images: remote` and the exact image alias can shift over time, particularly since the 2023 LXD/Incus project split. Confirm the alias before class with: `lxc image list images: | grep -i rocky`

If the check reports the container already exists, do not re-launch it — reuse the existing container for the rest of this section. Re-running `lxc launch` with the same name will error rather than silently replace it.

Step 3: Jump into the container

```
lxc exec os-demo-rocky -- bash

cat /etc/os-release
uname -r
ps aux
```

Inside the container: `/etc/os-release` shows Rocky Linux. `ps aux` shows `/sbin/init` as PID 1 — this is `systemd`; Rocky symlinks `/sbin/init` to the `systemd` binary, so the process name shown is the path it was invoked from, not the literal word "systemd".

Step 4: Return to the host

```
exit
```

Exiting the exec session does not stop the container — it keeps running, exactly like logging out of an SSH session to a real server.

1.4 Shared Kernel (Same Kernel, Different OS)

Run both commands on the host, then the same two inside the container, and put the results side by side.

Step 1: On the host

```
uname -r  
cat /etc/os-release
```

Step 2: Inside the container

```
lxc exec os-demo-rocky -- uname -r  
lxc exec os-demo-rocky -- cat /etc/os-release
```

Expected result: `uname -r` is IDENTICAL on host and container (same kernel — this is what makes it a container, not a VM). `/etc/os-release` is DIFFERENT (Ubuntu vs. Rocky — this is what makes it a genuinely separate OS-container environment). This contrast is the entire point of the demo — say both halves out loud.

Verify: `lxc list` shows `os-demo-rocky` as *RUNNING* throughout, confirming it survived the `exec`/`exit` cycle.

Part 1 Cleanup

```
lxc delete os-demo-rocky --force
```

Part 2 — Docker Demo

Goal: install Docker Engine, build a minimal custom image, and demonstrate the difference between detached and foreground container execution.

2.1 Install Docker

Step 1: Check whether Docker is already installed

```
if command -v docker &>/dev/null; then  
    echo "Docker is already installed:"  
    docker --version  
else  
    echo "Docker not found. Proceeding to install."  
fi
```

Step 2: Install Docker (only if the check above reports not found)

```
sudo apt update  
sudo apt install docker.io -y  
sudo systemctl enable --now docker
```

Verify: `docker --version` returns a version string; `docker ps` runs without a permission error (add your user to the `docker` group with `sudo usermod -aG docker $USER` and re-login if it does not). Alternative run all commands under `sudo`

2.1.1 Which Command Does What — Create, Start, Execute

The commands used across this document fall into three distinct categories, worth stating explicitly before running any of them: creating an image, creating/starting a container, and executing inside a container that is already running. These are not interchangeable, and confusing them is the source of the run vs. start naming-conflict issue covered later in 2.4.1.

Action	Command	What it actually does
Create an image	<code>docker build -t name:tag .</code>	Builds a new image from a Dockerfile
Create an image	<code>docker pull name:tag</code>	Downloads an existing image from a registry into local storage
Create an image	<code>docker commit <container> name:tag</code>	Snapshots a container's current state into a new image — generally discouraged; undocumented and unrepeatable compared to a Dockerfile
Create a container (not started)	<code>docker create --name X image:tag</code>	Makes a container from an image, in Created state — does not start it
Create AND start a container	<code>docker run --name X image:tag</code>	Combines create + start in one step — this is why run looks like it does everything at once
Start an existing container	<code>docker start X</code>	Resumes a container that already exists — creates nothing new
Execute inside a running container	<code>docker exec -it X <cmd></code>	Runs an additional command inside a container that is already running — does not create or start the container itself

`docker run` appears against both create and start because it genuinely does both in a single call. `docker create` on its own is the rarely-used, purer version of just the creation step, without immediately starting the result.

2.1.2 Worked Example: Create, Start, and Execute

A concrete walkthrough of the last row in the table above, using `alpine` directly rather than `simple-demo:1.0`, since that image is not built until 2.2.

Step 1: Create an image (pull)

```
docker pull alpine
```

Step 2: Create a container WITHOUT starting it

```
docker create --name demo-alpine alpine sleep 3600
docker ps -a
```

Verify: *demo-alpine* appears with *STATUS Created* — made, but never run.

Step 3: Start the container that already exists

```
docker start demo-alpine
docker ps
```

Verify: *demo-alpine* now shows *Up* — same container, same ID as step 2, nothing new was created.

Step 4: Execute inside the now-running container

```
docker exec -it demo-alpine sh

cat /etc/os-release
ps aux
exit
```

This confirms it is genuinely Alpine, and that `exec` opened a session inside the container that was already running from step 3 — it did not create or start anything new.

Step 5: Confirm the container is still running after exiting the `exec` session

```
docker ps
```

Verify: *demo-alpine still shows Up. Exiting an `exec` session does not stop the container — the same behaviour already seen with `lxc exec` in Part 1.*

Steps 2 and 3 together are exactly what `docker run --name demo-alpine alpine sleep 3600` would do in one call. This longer version exists to make the create/start split concrete before `docker run`'s single-step version is used everywhere else in this document.

Step 6: Cleanup

```
docker stop demo-alpine
docker rm demo-alpine
```

2.2 Create a Simple Image

A minimal image, kept deliberately small so the focus stays on the build/run mechanics rather than application code.

```
mkdir ~/simple-demo && cd ~/simple-demo
```

Step 1: Create the Dockerfile

```
FROM alpine:3.20
RUN echo 'Container built at:' > /message.txt && date >> /message.txt
CMD ["sh", "-c", "cat /message.txt && sleep 3600"]
```

Step 2: Build it

```
docker build -t simple-demo:1.0 .
```

Verify: *docker images shows simple-demo:1.0 in the list.*

2.2.1 How the Build Output Maps to Layers — Reading the Trace

Each Dockerfile instruction becomes its own temporary, throwaway container, which exists only long enough to produce one new, permanent, read-only image layer, then is deleted. This is visible directly in the build output, using a real trace as an example:

- Step 1/3 (`FROM alpine:3.20`) — no container runs; Docker just references the pulled base layer directly, e.g. ---> d9e853e87e55
- Step 2/3 (`RUN echo ... > /message.txt`) — Docker starts a temporary container (e.g. Running in d247dd4cfba6), executes the instruction inside it, commits the filesystem change as a new layer (e.g. ---> 97aa002b981e), then deletes the temporary container (Removed intermediate container d247dd4cfba6)
- Step 3/3 (`CMD [...]`) — same pattern again, even though `CMD` only records metadata rather than changing any files: a new temporary container (e.g. Running in 95a124946b71), a new layer (e.g. ---> d1cab55c3cf7), then deletion of the temporary container

The final line, `Successfully built <id>`, is simply the ID of the very last layer produced — that ID becomes the image, and `Successfully tagged simple-demo:1.0` attaches the human-readable name to it. Three instructions produced a straight chain of three layer IDs, each built directly on top of the last — that chain is the image.

Caution: This linear "Step N/M ... Running in ... Removed intermediate container" trace is the classic/legacy Docker builder output. Modern Docker has defaulted to BuildKit since Engine 23.0, which displays build progress differently (a [+] Building tree view, not this linear trace). If a student's own machine shows a different-looking build log, the underlying layer-per-instruction mechanism is identical — only the display format differs.

Step 3: Confirm the image ID directly

```
docker images
```

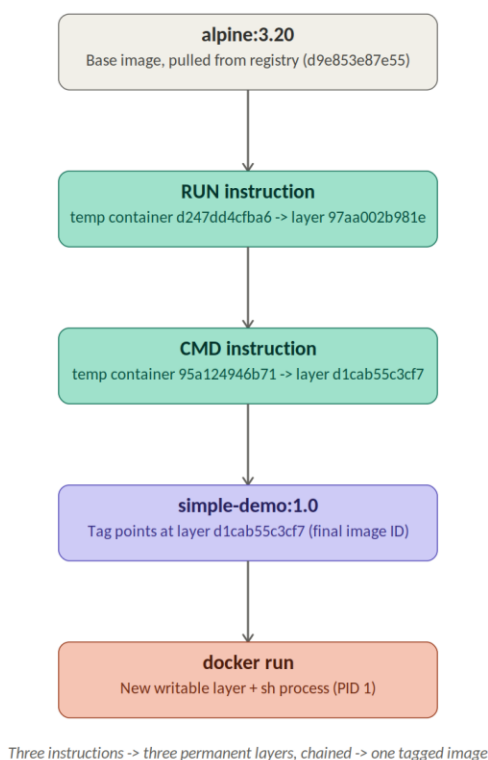
The IMAGE ID column shows the same (truncated) value as the Successfully built line. To see the full, untruncated ID:

```
docker images --no-trunc
```

Step 4: See the full layer chain as a permanent, inspectable property of the image

```
docker history simple-demo:1.0
```

This lists every layer in the image, most recent first — the same FROM/RUN/CMD chain from the build trace, now visible after the fact rather than only during the build itself.



Visual trace of the exact build used above: one base image, two new layers (RUN, CMD), one final tag, one running container with its own writable layer on top. Note the temporary container IDs (d247dd4cfba6, 95a124946b71) do not appear anywhere in the final artifact — they existed only long enough to produce the layer beside them, then were deleted.

The real terminal output for this exact image, with full untruncated hashes, is worth showing directly rather than only the abstracted diagram above:

```
docker history simple-demo:1.0 --no-trunc
```

IMAGE	CREATED	CREATED BY	SIZE
d1c208c40fe0...	4 days ago	CMD ["sh" "-c" "cat /message.txt && sleep 3600"]	0B
c52831d18f02...	4 days ago	echo 'Container built at:' > ... && date >> ...	8.19kB
d9e853e87e55...	4 months ago	CMD ["/bin/sh"] (alpine:3.20's own CMD, inherited)	0B
<missing>	4 months ago	ADD alpine-minirootfs-3.20.10-x86_64.tar.gz /	8.47MB

The top two rows are this Dockerfile's own instructions — the 8.19kB RUN layer is the real, physical size of the RUN instruction's filesystem change; the CMD row is 0B, metadata only, exactly as predicted. The bottom two rows belong to alpine:3.20 itself, inherited wholesale because simple-demo is FROM alpine:3.20 — not re-executed, just carried forward as part of the ancestry.

```
root@ip-172-31-15-162:/home/ubuntu/simple-demo# docker history simple-demo:1.0 --no-trunc
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
sha256:d1c208c40fe0dc319be1a43fcedefcc8e84c4642e453701a6fb3c3a948980824 4 days ago       /bin/sh -c #(nop)  CMD ["sh" "-c" "cat /message.txt && sleep 3600"]      0B
sha256:c52831d18f025bc9df933cc9c1a51a42d626bbe5f8afac96028bfa83a7d419 4 days ago       /bin/sh -c echo 'Container built at:' > /message.txt && date >> /message.txt 8.19kB
sha256:d9e853e87e5526f6b2917df91a2115c36dd7c696a35be12163d4de6e2a4b0bc 4 months ago     CMD ["/bin/sh"]          0B      buildkit.dockerfile.v0
<missing>      4 months ago     ADD alpine-minirootfs-3.20.10-x86_64.tar.gz / # buildkit          8.47MB   buildkit.dockerfile.v0
root@ip-172-31-15-162:/home/ubuntu/simple-demo#
```

2.2.1a Why the Alpine Layer Shows <missing> Instead of a Hash

This is real, documented Docker behaviour, not a broken or corrupted image — confirmed by an issue filed directly against BuildKit (moby/buildkit#1235, "Image id shown as missing with docker history").

Docker only assigns and displays a proper image ID for a layer when that layer is the final, distinct output of its own separately-built-and-registered image. alpine:3.20's own CMD ["/bin/sh"] row gets a real ID (d9e853e87e55...) because that is the top layer of alpine:3.20 as its own separately pulled and tagged image. The ADD alpine-minirootfs... row sits one level below that, inside Alpine's own internal build chain — it was never itself registered as a standalone image with its own ID, only as an intermediate step on the way to Alpine's final tagged layer. Once an image is distributed through a registry (exactly how alpine:3.20 arrived on this instance, via docker pull), that internal intermediate-step bookkeeping is not preserved or transmitted — only the layer content and the final image ID survive the trip.

Caution: <missing> describes a bookkeeping gap, not missing data. The 8.47MB of actual Alpine filesystem content is still fully present and in use — Docker is simply no longer tracking that particular intermediate step as its own addressable image.

2.2.2 Why Both alpine:3.20 and simple-demo:1.0 Appear in docker images

docker build never consumes or deletes the base image — it only adds new layers on top of it. alpine:3.20 has to already exist locally (pulled automatically the first time a Dockerfile references it, if not already cached) as its own independently-tagged image. simple-demo:1.0 is a new tag pointing at the final layer produced by the build, but that layer's entire ancestry still includes alpine:3.20's original layers underneath it, shared rather than duplicated.

Verify this directly rather than taking it on faith:

```
docker history alpine:3.20
docker history simple-demo:1.0
```

The bottom rows of simple-demo:1.0's history show the same layer IDs as alpine:3.20's own history — proof they are the same physical data on disk, not two copies. docker images lists tags, not raw storage; two tags can share an ancestry without doubling disk usage.

2.2.3 Does docker build Push to a Registry Automatically? No

Every image built or run across this document — simple-demo, course-postgres, student-query-app — has existed only in this one EC2 instance's local image store. If the instance were terminated right now, all of them

would simply be gone. Pushing to a registry is a separate, deliberate action, not something docker build or docker run ever does on their own:

```
docker login
docker tag simple-demo:1.0 <your-dockerhub-username>/simple-demo:1.0
docker push <your-dockerhub-username>/simple-demo:1.0
```

The tag must be renamed to include a namespace (a username, or a private registry's hostname) before pushing — docker push simple-demo:1.0 alone fails, since Docker Hub has no way to know whose repository it belongs in.

2.3 Run in Foreground (Non-Detached) Mode

```
docker run -it --name demo-foreground simple-demo:1.0
```

This blocks the terminal and prints the container's output directly. Press Ctrl+C to stop it — point out that this also stops the container, since the terminal session was attached to the container's main process.

Caution: If Ctrl+C appears to do nothing and the container keeps running, this is expected behaviour with this exact Dockerfile, not a mistake. See 2.3.1 below before assuming something is broken.

```
docker ps -a
```

Verify: *demo-foreground* shows *STATUS Exited*.

2.3.1 If Ctrl+C Does Not Stop the Container — the PID 1 Signal Problem

This Dockerfile's CMD is ["sh", "-c", "cat /message.txt && sleep 3600"], which puts sh at PID 1 inside the container. Linux gives PID 1 special treatment: signals that would normally terminate a process (SIGINT, SIGTERM) are silently ignored unless PID 1 explicitly installs a handler for them. Plain sh running a script does not. Ctrl+C's SIGINT is effectively swallowed, and the container never exits.

Immediate escape, from a second terminal/SSH session, if this happens live:

```
docker kill demo-foreground
```

SIGKILL cannot be ignored or blocked by any process, PID 1 or not — this always works immediately. docker stop would also eventually work, but only after waiting out a 10-second grace period and falling back to SIGKILL itself.

The robust fix, for a container whose main process is not a real application that can install its own signal handlers:

```
docker run -dit --init --name demo-foreground simple-demo:1.0
```

The --init flag injects a small, purpose-built init process (tini) as the container's real PID 1. Tini correctly handles signals and forwards them to its child — and since sleep is then no longer PID 1 itself, the kernel's normal default behaviour applies to it: SIGTERM/SIGINT terminate it immediately, no handler required. Exec-form CMD (avoiding shell-form entirely) helps, but does not fully solve this on its own — the command still needs either --init/tini, or its own signal handling, since being PID 1 alone does not guarantee a process reacts to signals correctly.

2.4 Run in Detached Mode

```
docker run -dit --name demo-detached simple-demo:1.0
```

Control returns immediately. The container keeps running in the background.

```
docker ps
docker logs demo-detached
```

Verify: *demo-detached* shows STATUS Up, and *docker logs* prints the same message without needing to attach to the container.

This is the same detached/foreground distinction that matters later in Part 3 and was covered when the Postgres demo was first designed — worth restating here explicitly since it is easy to lose a container by running it in foreground mode and pressing Ctrl+C without realizing what happened.

2.4.1 docker run vs. docker start — Why This Matters for Reused Containers

A common error when reusing an instance across sessions: `docker run --name X ...` fails with "Conflict. The container name ... is already in use" if a container with that name already exists, running or stopped. `docker ps` only shows running containers; `docker ps -a` shows every container regardless of state, and is the correct first check whenever a name conflict appears.

	docker run	docker start
Acts on	An image — always creates a new container	An existing container, by name or ID
Result	A new container ID every time	Same container, same ID, same prior filesystem state
<code>-p / -v / -e / --name</code>	Accepted — creation-time settings	Not accepted — fixed when the container was first created
Fails when	The name is already in use (running or stopped)	The container does not exist at all

Rule of thumb: if the container does not exist yet, use `run`. If it already exists (check with `docker ps -a`) and you only need it running again, use `start` — running again with the same name will always collide.

```
docker rm -f demo-foreground
docker run -it --name demo-foreground simple-demo:1.0

# — or, to resume the same container instead of recreating it —
docker start -ai demo-foreground
```

Caution: `docker start` alone (no flags) does not reattach your terminal, regardless of how the container was originally created with `-it`. It only resumes the container's process running in the background and returns your prompt immediately — the container is not behaving differently, nothing is attached to watch it. Use `docker start -ai <name>` specifically (`-a` attaches STDOUT/STDERR, `-i` keeps STDIN interactive) to reattach and see the exact same behaviour `docker run -it` gave the first time, including the Ctrl+C/PID-1 signal issue from 2.3.1 if it is still present.

Part 2 Cleanup

```
docker rm -f demo-foreground demo-detached
```

Part 3 — Dockerfile Demo (PostgreSQL)

Goal: contrast running an official image with no Dockerfile against extending that image with a custom Dockerfile; build a second, real application image (a Python client) that consumes that database safely over a private Docker network; then, separately, connect from an external client (your laptop) to demonstrate real public network access — not just `docker exec`.

3.1 Docker Demo Using the Actual Official Image — No Dockerfile

Step 1: Pull the image explicitly — make the registry interaction visible

`docker run` would pull this image silently in the background on first use if it is not already cached. Pulling it explicitly, as its own step, surfaces the registry interaction that would otherwise be invisible.

```
docker pull postgres:16
```

Walk through the actual output:

- `library/postgres` — `library/` is Docker Hub's namespace specifically for Official Images, the same category covered on the Registries slide
- Each line below the first (a hash followed by "Pull complete") is a separate layer, pulled independently with its own hash — the same layer concept from the simple-demo build trace in 2.2.1, now running in reverse: downloading existing layers instead of creating new ones
- The `Digest:` line is the image's content hash as it exists on the registry — the actual proof of where the image came from, not just a trusted filename

Step 2: Confirm the registry origin as a permanent property of the local image

```
docker image inspect postgres:16 --format='{{.RepoDigests}}'
```

Prints the exact registry and digest this image was pulled from — verifiable after the fact, the same way `docker history` made the build layers inspectable after the build finished.

Step 3: Create the volume and run the container

```
docker volume create pgdata

docker run -d \
  --name pg-demo \
  -e POSTGRES_PASSWORD=demoPass123 \
  -e POSTGRES_DB=coursedb \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:16
```

Verify: `docker ps` shows `pg-demo` with `STATUS Up`. `docker exec -it pg-demo psql -U postgres -d coursedb -c "SELECT version();" returns the PostgreSQL version string.`

3.2 Docker Demo Using a Dockerfile — Extending the Base Image

This is the same Postgres extension pattern discussed earlier in the course: a Dockerfile that adds exactly one meaningful instruction to a well-built base image, rather than building from scratch.

Step 1: Create the project folder and files

```
mkdir ~/pg-custom && cd ~/pg-custom
```

Step 2: Dockerfile

```
FROM postgres:16
COPY init.sql /docker-entrypoint-initdb.d/
```

Step 3: `init.sql`

```
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    course TEXT NOT NULL  
);  
  
INSERT INTO students (name, course) VALUES  
    ('Arun', 'Cloud Computing'),  
    ('Demo Student', 'Cloud Computing');
```

Step 4: Build and run on a different host port (5433) to avoid clashing with pg-demo from 3.1

```
docker build -t course-postgres:1.0 .  
docker volume create pgdata-custom  
docker run -d \  
    --name pg-course-demo \  
    -e POSTGRES_PASSWORD=demoPass123 \  
    -v pgdata-custom:/var/lib/postgresql/data \  
    -p 5433:5432 \  
    course-postgres:1.0
```

Verify: `docker exec -it pg-course-demo psql -U postgres -d postgres -c "SELECT * FROM students;"` returns the two seeded rows automatically — proving the `init.sql` ran on first container start.

3.2.1 What a Volume Actually Is, and What -v Instructs

Postgres's own official Dockerfile already declares VOLUME [/var/lib/postgresql/data] internally — visible directly in its layer history below. If `pgdata-custom` were never explicitly created, Docker would still honour that declaration, but by creating an anonymous volume with an auto-generated hash for a name instead of a human-readable one. Naming it explicitly does not add a capability that was not already there structurally — it makes an existing, otherwise-anonymous capability manageable: findable, reattachable to a fresh container later, and identifiable among other volumes on the host.

Where it physically lives, and what "type" it is — check directly rather than assume:

```
docker volume inspect pgdata-custom
```

On a standard Ubuntu Docker install, the Mountpoint field reads `/var/lib/docker/volumes/pgdata-custom/_data` — an ordinary directory on the host's own root filesystem, nothing exotic. The Driver field reads `local`, the default driver; a network-backed alternative (e.g. NFS) is possible via the same local driver's options, not a different driver entirely.

What `-v pgdata-custom:/var/lib/postgresql/data` instructs is Docker, not Postgres. Postgres has no awareness this mount exists — it simply reads and writes to `/var/lib/postgresql/data` as an ordinary path, using the `PGDATA` environment variable declared inside its own image (visible in the layer trace below). Before the container's process starts, Docker performs a bind mount: the host directory `/var/lib/docker/volumes/pgdata-custom/_data` is mounted at that path inside the container's filesystem view, using the same mount-namespace mechanism covered earlier in this course — the container's process believes it has an ordinary local directory, while every read and write is transparently redirected to the host-side location. This is precisely why removing a container never touches this data: the mounted host directory was never part of the container's own temporary writable layer to begin with.

Caution: `docker volume create` is not strictly required. `docker run -v pgdata-custom:...` alone creates the volume automatically with the local driver if it does not already exist. The explicit create step in this document is a deliberate teaching choice — making volume creation a visible, separate action — not a technical requirement.

3.2.2 The Real Layer Trace for course-postgres:1.0

Only one instruction in this Dockerfile is actually new — everything else below it belongs to postgres:16's own build history, inherited wholesale, the same mechanism already confirmed with alpine:3.20 in 2.2.1a.

```
docker history course-postgres:1.0
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
281c4e011d44	3 minutes ago	/bin/sh -c #(nop) COPY file:306b1367289b2b62...	12.3kB	
f1c3376c26f2	2 weeks ago	CMD ["postgres"]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	EXPOSE map[5432/tcp:{}]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	STOPSIGNAL SIGINT	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENTRYPOINT ["docker-entrypoint.sh"]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c ln -sT docker-ensure-initdb.s...	16.4kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	COPY docker-entrypoint.sh docker-ensure-init...	36.9kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	VOLUME [/var/lib/postgresql/data]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c install --verbose --directory...	20.5kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV PGDATA=/var/lib/postgresql/data	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c install --verbose --directory...	12.3kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; dpkg-divert --add ...	106kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -ex; export PYTHONDONTW...	336MB	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV PG_VERSION=16.15-1.pgdg13+2	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV PATH=/usr/local/sbin:/usr/local/bin:/usr...	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV PG_MAJOR=16	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -ex; key='B97B0AFCAA1A47...	24.6kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c mkdir /docker-entrypoint-init...	8.19kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; apt-get update; a...	3.83MB	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV LANG=en_US.utf8	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; if [-f /etc/dpkg/...	27.3MB	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; savedAptMark="\$(ap...	3.74MB	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV GOSU_VERSION=1.19	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -ex; apt-get update; ap...	16.9MB	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; groupadd -r postgr...	53.2kB	buildkit.dockerfile.v0

The top row (COPY file:306b1367289b2b62..., 12.3kB) is the only instruction this Dockerfile actually ran — copying init.sql in. Every row below it, timestamped weeks earlier, is postgres:16's own official build: creating the postgres system user, installing GPG keys to verify the PostgreSQL apt repository, running apt-get to install PostgreSQL itself, declaring VOLUME [/var/lib/postgresql/data] (the exact declaration referenced in 3.2.1 above), and installing docker-entrypoint.sh together with docker-ensure-initdb.sh — the literal script responsible for the /docker-entrypoint-initdb.d/ behaviour that makes init.sql run automatically on first startup.

Almost every row shows <missing> for the same reason established with alpine:3.20 — only the final, top layer of postgres:16 as its own separately-tagged image (CMD ["postgres"]) carries a real image ID. Every step inside Postgres's own build chain below that was never itself individually registered as a standalone local image.

Worth pointing out the scale contrast directly: this Dockerfile's own contribution is 12.3kB. The single largest layer in Postgres's own build — installing the core server binaries — is 336MB. The base image being extended is, by a wide margin, the majority of what is actually running, even though the Dockerfile itself is two lines long.

3.3 Python Client Demo — Querying PostgreSQL from a Custom Image

Extends the pg-course-demo container from 3.2 with a separate Python application container that connects to it, queries the students table, and prints the results. This section demonstrates three things together:

dependency-before-code layering in a real Dockerfile, excluding files from the build context, and passing database credentials to a container without hardcoding them.

3.3.1 Project Files

Step 1: Create the project folder

```
mkdir ~/student-query-app && cd ~/student-query-app
```

Step 2: requirements.txt

```
psycopg2-binary==2.9.9
```

Step 3: query_students.py

```
import os
import psycopg2

conn = psycopg2.connect(
    host=os.environ["DB_HOST"],
    port=os.environ.get("DB_PORT", "5432"),
    dbname=os.environ["DB_NAME"],
    user=os.environ["DB_USER"],
    password=os.environ["DB_PASSWORD"],
)

cur = conn.cursor()
cur.execute("SELECT id, name, course FROM students;")
rows = cur.fetchall()

print("Students enrolled:")
for row in rows:
    print(f" {row[0]}: {row[1]} - {row[2]}")

cur.close()
conn.close()
```

Every credential is read from `os.environ` — nothing is hardcoded in the script, which is what makes 3.3.4 possible.

3.3.2 Dockerfile — Dependencies Before Code

Step 1: Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY query_students.py .

RUN useradd -m appuser
USER appuser

CMD ["python3", "query_students.py"]
```

Important Note:

- requirements.txt is copied and installed BEFORE query_students.py is copied.
- This is the layer-caching order from the Do's/Don'ts slide, now shown in a Dockerfile that has a real reason to care about it — editing the Python script later will not force psycopg2 to reinstall.

Step 2: USER appuser

The container runs as a non-root user by the time CMD executes — direct application of the earlier 'don't leave the container running as root' rule.

3.3.3 .dockerignore

Mock files are listed here specifically to demonstrate exclusion — they don't need to actually exist for the teaching point to land, but creating them for the demo makes the effect visible.

```
.git
__pycache__/
*.pyc
notes.txt
local_test_data.csv
.env.local
venv/
```

Optional: create a couple of these files (touch notes.txt local_test_data.csv) before building, then show docker build with and without the .dockerignore file present to demonstrate the build context size difference directly.

3.3.4 Build, Tag, and Run — Credentials Injected at Runtime

Note on scope: student-query-app is intentionally stateless — it connects, runs one query, prints the result, and exits. It does not get its own volume. It reads from pgdata-custom (created in 3.2), via the already-running pg-course-demo container, over the pgnet network. Volumes belong to containers that own data; this container only reads it.

Step 1: Build and tag the image

```
docker build -t student-query-app:1.0 .
```

Step 2: Confirm pg-course-demo is still running before proceeding

```
docker ps | grep pg-course-demo
```

If this returns nothing, pg-course-demo has been removed or was never restarted since a prior session. The DB_HOST=pg-course-demo lookup in Step 4 depends on Docker's embedded DNS resolving that container name — if the container doesn't exist, the query app will fail to connect, not because of a volume or credentials problem, but because the name it's trying to reach isn't there. Re-run the docker run command from 3.2 to bring it back before continuing.

Step 3: Create a private Docker network and attach the existing Postgres container

```
docker network create pgnet
docker network connect pgnet pg-course-demo
```

Step 4: Verify the network connection is real before trusting the query app's success

```
docker network inspect pgnet
```

Shows both containers attached to the network with their actual internal IPs, assigned by Docker — confirms the network genuinely exists and has a member, rather than trusting the next step's success alone.

```
docker run --rm --network pgnet alpine getent hosts pg-course-demo
docker exec pg-course-demo hostname -i
```

Verify: The first command proves Docker's embedded DNS resolves the container name `pg-course-demo` to a real internal IP. The second shows that container's actual IP directly. The two values should match — that is the concrete, checkable proof the name resolution mechanism is genuinely working, independent of whether the query app happens to succeed afterward.

Step 5: Run the query container on that network, with credentials injected at runtime

```
docker run --rm --network pgnet \
  -e DB_HOST=pg-course-demo \
  -e DB_PORT=5432 \
  -e DB_NAME=postgres \
  -e DB_USER=postgres \
  -e DB_PASSWORD=demoPass123 \
  student-query-app:1.0
```

Verify: The container prints the two seeded rows from the students table (Arun and Demo Student) and then exits (`--rm` removes it automatically).

Caution: Two points worth stating precisely rather than as an absolute "safe" claim. First: credentials are passed at runtime, never written into the Dockerfile or baked into an image layer — this is strictly better than hardcoding, since a hardcoded value is permanent, travels with the image everywhere it's pushed or pulled, and is visible to anyone who has the image even without running it (docker history `--no-trunc` shows it in plaintext). Second: `DB_HOST=pg-course-demo` resolves via Docker's embedded DNS on the shared `pgnet` network, so this connection never leaves the host and does not require opening the security group at all — the safer of the two access patterns in Part 3, contrasted with 3.4 below. Neither runtime method here is fully secure in an absolute sense: the credential is still visible in plaintext via `docker inspect` on this container while it runs. Treat this hierarchy as Dockerfile-hardcoded (worst) → `export + -e` (better, but leaves a `bash_history` trail) → `--env-file` (better still, no history trail, still plaintext on disk) — real production secret handling is a separate, dedicated topic outside this demo's scope.

3.3.5 Setting Up These Environment Variables on the EC2 Host

Caution: Setting a variable on the EC2 host with `export` does NOT automatically make it available inside the container. The container has its own isolated environment namespace — the same principle covered earlier for process namespaces applies here to environment variables too. Docker must be explicitly told to forward a host variable into the container.

Method 1 — Host-Level Export, Explicitly Forwarded

```
export DB_HOST=pg-course-demo
export DB_PORT=5432
export DB_NAME=postgres
export DB_USER=postgres
export DB_PASSWORD=demoPass123

docker run --rm --network pgnet \
  -e DB_HOST \
  -e DB_PORT \
  -e DB_NAME \
  -e DB_USER
```

```
-e DB_PASSWORD \
student-query-app:1.0
```

Point out the syntax difference: `-e DB_HOST` with no `=value` tells Docker to pull that variable's current value from the calling shell and forward it into the container — different from `-e DB_HOST=value` used in the 3.3.4 run command above.

Caution: Credentials typed via `export` tend to end up in `~/.bash_history` and remain readable from `/proc/<pid>/environ` for the life of the shell session. Acceptable for a demo; not recommended for real credential handling.

Method 2 — `--env-file` (Recommended Practice)

```
cd ~/student-query-app
cat > db.env << 'EOF'
DB_HOST=pg-course-demo
DB_PORT=5432
DB_NAME=postgres
DB_USER=postgres
DB_PASSWORD=demoPass123
EOF
chmod 600 db.env

docker run --rm --network pgnet --env-file db.env student-query-app:1.0
```

`chmod 600` restricts the file to owner-only read/write. This keeps credentials out of shell history, makes the run command reusable without retyping every flag, and reflects how this is actually handled in practice — `db.env` belongs in `.gitignore` and should never be committed alongside the Dockerfile.

Step 1: Cleanup

```
rm ~/student-query-app/db.env
```

Remove the credentials file once the demo is complete, the same way the security group rule in 3.4 gets closed afterward — neither should persist beyond the session.

3.4 Security Group Configuration for Laptop Access

Caution: Opening a database port to the internet is a real exposure. Scope the inbound rule to your current IP address only, never `0.0.0.0/0`, and close the rule again once the demo is finished. This applies to the SSH rule as well — leaving any rule on `0.0.0.0/0`, SSH included, exposes the instance to constant automated scanning.

Step 1: Find your current public IP

Use "What is my IP" in a browser, or:

```
curl https://checkip.amazonaws.com
```

Step 2: Edit the EC2 security group

- AWS Console → EC2 → Security Groups → select the instance's security group
- Edit inbound rules → Add rule
- Type: PostgreSQL (auto-fills port 5432) or Custom TCP, port 5432 (and 5433 if connecting to the custom container as well)
- Source: "My IP" (AWS auto-fills your current address as a `/32`) — do not select `Anywhere/0.0.0.0/0`
- Save rules

Caution: "My IP" is a snapshot taken at the moment the rule is saved, not a live setting. If your laptop's public IP changes later (different network, VPN toggle), the rule stops matching and the connection will fail — check your current IP again before assuming something else broke.

Step 3: Connect from your laptop

Using a local client (psql, DBeaver, pgAdmin, or similar), connect to:

```
Host: <ec2-public-ip>
Port: 5432 (or 5433 for the custom image)
User: postgres
Password: demoPass123
Database: coursedb (or postgres, for the custom image)
```

Verify: The client connects successfully and the students table (from 3.2) is visible if connecting to port 5433.

3.4.1 Troubleshooting: DBeaver "invalid value for parameter TimeZone: Asia/Calcutta"

This is a known, actively-tracked bug in the PostgreSQL JDBC driver, not a problem with the EC2 instance, Docker, or Postgres itself. At connection time, DBeaver's JDBC driver reads the client machine's own JVM default timezone and sends it to Postgres as a startup parameter. On many Windows machines with an India-region locale, that JVM resolves to the legacy identifier Asia/Calcutta rather than the modern Asia/Kolkata (the IANA database renamed this zone in 2006). The postgres:16 image does not recognise the old alias and rejects the connection outright, before login even occurs.

Step 1: Fix — override the timezone DBeaver sends

- Right-click the connection in DBeaver → Edit Connection → Driver properties tab
- Add property: TimeZone = Asia/Kolkata
- Press Enter in the value field to commit it, then click OK/Finish on the whole dialog (not Cancel) — a property that is typed but never committed will silently fail to save

Step 2: If the property will not save — set it at the JVM level instead

Edit dbeaver.ini (normally alongside dbeaver.exe, commonly C:\Program Files\DBeaver\dbeaver.ini) and add under the -vmargs section:

```
-Duser.timezone=Asia/Kolkata
```

Caution: If DBeaver is installed under Program Files, editing this file requires admin rights — open the text editor as Administrator, or the edit will silently fail to save, the same symptom as the in-app property not saving.

Step 3: Confirm the fix

```
SHOW TIMEZONE;
```

Verify: Returns Asia/Kolkata, not Asia/Calcutta.

3.4.2 Optional: Prove the Connection Is Live, Not Cached

Stopping and restarting the same container while DBeaver is connected demonstrates that the connection genuinely reaches a live process on the EC2 instance.

Step 1: Stop the container

```
docker stop pg-course-demo
```

Attempt to reconnect or run a query in DBeaver — it should fail with a connection error, since nothing is listening on the port anymore.

Step 2: Restart the same container (not docker run)

```
docker start pg-course-demo
```

Caution: Use `docker start` here, not `docker run` — see 2.4.1. `docker run` would try to create a new container and immediately collide on the existing name.

Step 3: Confirm and reconnect

```
docker ps
```

Verify: `pg-course-demo` shows Up again. Reconnecting DBeaver with the same settings succeeds, and `SELECT * FROM students;` still returns both seeded rows — proving that a stop/start cycle (unlike remove-and-recreate) never touches the container's data at all.

Step 4: Close the port after the demo

Remove or disable the inbound rule added above as soon as the demo is complete. Do not leave a database port open on a shared teaching instance between sessions.

Part 3.5 — Cleaner Alternative: Docker Compose

3.2 and 3.3 together demonstrate the same thing Compose does in one file: a database container and a client container, connected over a private network, without hardcoded credentials. The manual `docker network create/connect` steps in 3.3 are what Compose automates. This section is not a replacement for 3.2/3.3 — the manual version is what actually teaches what Compose is doing underneath.

Caution: A single Dockerfile with two `FROM` instructions does NOT combine two services into one image or one container. Each `FROM` starts an entirely new, independent build stage, discarding everything before it unless explicitly carried forward with `COPY --from=<stage>`. A container also has one filesystem and one main process — there is no way for a single container to genuinely be both a full Postgres server and a Python client at once. Compose keeps the two Dockerfiles genuinely separate and connects them at runtime, which is the correct version of "merging" this demo.

Step 1: Project layout

```
~/compose-demo/
docker-compose.yml
pg-custom/
  Dockerfile      (from 3.2)
  init.sql        (from 3.2)
student-query-app/
  Dockerfile      (from 3.3)
  requirements.txt (from 3.3)
  query_students.py (from 3.3)
```

Step 2: docker-compose.yml

```
services:
  db:
    build: ./pg-custom
    environment:
      POSTGRES_PASSWORD: demoPass123
    ports:
```

```
- "5433:5432"
volumes:
  - pgdata-custom:/var/lib/postgresql/data

query-app:
  build: ./student-query-app
  depends_on:
    - db
  environment:
    DB_HOST: db
    DB_PORT: 5432
    DB_NAME: postgres
    DB_USER: postgres
    DB_PASSWORD: demoPass123

volumes:
  pgdata-custom:
```

DB_HOST: db works because Compose automatically makes each service reachable by its service name on a network it creates for you — no manual docker network create/connect needed, directly replacing the pgnet steps from 3.3.

Caution: ports: ["5433:5432"] is included here specifically so DBeaver/laptop access (3.4) still works against the Compose-managed database. This is a deliberate choice, not a default — the manual pgnet version in 3.3 intentionally avoided any public exposure at all, since query-app never needed it. If laptop access to this Compose stack is not required for a given session, remove the ports line entirely and rely on docker compose exec db psql instead, keeping the same no-exposure posture as 3.3.

Step 3: Build and run everything with one command

```
cd ~/compose-demo
docker compose up --build
```

Verify: Both containers start; query-app's logs show the same two seeded rows printed once, then the container exits (it is not a long-running service).

Step 4: Cleanup

```
docker compose down -v
```

The -v flag also removes the named volume — omit it if the seeded data should persist for a later session.

Part 3 Cleanup

```
docker rm -f pg-demo pg-course-demo
docker network rm pgnet
docker rmi student-query-app:1.0 course-postgres:1.0
rm -f ~/student-query-app/db.env
```

The query container itself does not need manual removal — it was run with --rm and cleaned itself up on exit.

Appendix — Command Glossary

Every command and flag used across this document, grouped by tool and purpose. Flags not used anywhere in this document are omitted deliberately, to keep this a reference for what was actually run rather than a full manual.

LXD / LXC

Command	Flags used	Meaning
snap install lxd	—	Installs LXD via snap, the current recommended method
lxd init --minimal	--minimal	Initializes LXD non-interactively with default settings
lxc launch <image> <name>	—	Creates and starts an LXC container from an image in one step
lxc exec <name> -- <cmd>	--	Runs a command inside a running container; -- separates lxc's own flags from the command being executed
lxc list	--format csv -c n	Lists containers; narrows output to just the name column, for scripting
lxc delete <name>	--force	Deletes a container; allows deleting one that is still running
lxc image list images:	—	Lists images available from the images: remote
lxc image info <image>	—	Shows metadata for one image — size, fingerprint, aliases
lxc remote list	—	Shows which URL each configured remote actually resolves to
lxc info <name>	—	Shows container details, including its real host-side PID

Docker — Images

Command	Flags used	Meaning
docker pull <image>	—	Downloads an image from a registry into local storage
docker build -t <name>:<tag> .	-t	Builds an image from a Dockerfile; sets its name and tag
	--target <stage>	Builds only up to and including a named stage in a multi-stage Dockerfile
	--no-cache	Ignores cached layers, forces a full rebuild
	--progress=plain	Shows the full, unabridged build log instead of BuildKit's collapsed tree view
	-f <file>	Uses a Dockerfile with a non-default name or path
	--secret id=X,env=Y	Supplies a BuildKit secret for RUN --mount=type=secret steps
	--build-arg	Passes a build-time ARG value
docker images	--no-trunc	Lists local images; shows full, untruncated image IDs
docker image inspect <image>	--format='{{.RepoDigests}}'	Shows the exact registry and digest an image was pulled from
docker history <image>	--no-trunc	Lists every layer in an image with size and originating instruction
docker tag <src> <dst>	—	Creates a new tag pointing at an existing image, often to add a registry namespace before pushing
docker push <image>	—	Uploads a tagged image to a registry
docker login	—	Authenticates to a registry
docker rmi <image>	—	Removes a local image

Docker — Container Lifecycle

Command	Flags used	Meaning
docker create --name X <image>	--name	Creates a container without starting it
docker run <image>	-d	Detached — runs in the background, returns control immediately
	-it	Interactive plus a pseudo-TTY; keeps a terminal attached

	-dit	Detached but still interactive/TTY-allocated — used so it can be attached to later
	--name	Assigns a container name
	-e KEY=VALUE	Sets an environment variable at runtime
	-v <vol>:<path>	Mounts a named volume (or bind mount) into the container
	-p <host>:<container>	Publishes a container port to the host
	--network <net>	Attaches the container to a specific Docker network at creation
	--rm	Automatically removes the container when it exits
	--init	Injects tini as PID 1 so signals are handled and forwarded correctly
	--env-file <file>	Loads environment variables from a file instead of individual -e flags
docker start <name>	-a	Attaches STDOUT/STDERR when starting an existing container
	-i	Keeps STDIN open (interactive) when starting
docker stop <name>	—	Sends SIGTERM, waits a grace period, then SIGKILL if still running
docker kill <name>	—	Sends SIGKILL immediately — cannot be ignored or blocked
docker rm <name>	-f	Removes a container; forces removal even if it is still running
docker ps	-a	Lists containers; shows all states, not just running
docker exec <name> <cmd>	-it	Runs an additional command inside an already-running container, interactively
docker logs <name>	—	Shows a container's captured stdout/stderr output

Docker — Networks & Volumes

Command	Meaning
docker network create <name>	Creates a private, user-defined network with built-in DNS resolution by container name
docker network connect <net> <container>	Attaches an already-running container to a network
docker network inspect <net>	Shows all containers attached to a network and their internal IPs
docker volume create <name>	Creates a named, Docker-managed persistent volume
docker volume inspect <name>	Shows a volume's storage driver and host filesystem location

Docker — System & Diagnostics

Command	Flags used	Meaning
docker system df	-v	Shows disk usage; breaks it down per image/container with SHARED/UNIQUE size
command -v docker	—	Shell check for whether a command exists (used in the idempotent install checks)
systemctl is-active <service>	--quiet	Checks whether a systemd service is running, suppressing output

Docker Compose

Command	Flags used	Meaning
---------	------------	---------

docker compose up	--build	Builds images (if needed) and starts every service defined in docker-compose.yml
docker compose down	-v	Stops and removes containers/networks; also removes named volumes